

Documenting Research Software

October 6, 2026

Contents

About this guide	1
Normative language and the ladder	2
Terminology	2
At a glance	3
1 What to document	3
1.1 Diátaxis: tutorials, how-to guides, reference, explanation	3
1.2 Grow the documentation with the software	4
1.3 The README is the front door	4
1.4 The paper is not the manual	4
2 Writing documentation	5
2.1 The reference lives next to the code	5
2.2 Document every key and every file	5
2.3 Comments say why, not what	6
2.4 Keep a changelog	6
2.5 Credit where credit is due	6
3 Keeping documentation true	6
3.1 Docs as code	6
3.2 Every example runs	7

About this guide

This guide is about documenting research software: organizing, writing, and maintaining what users and maintainers read besides the code. Together with *Design Principles for Research Software*, cited below as *Design Principles*, and *Testing Research Software*, it forms one set of guidelines for all code written for a research purpose, in academia or in industry, in any programming language and scientific field. It is meant for looking things up while working, not for reading from cover to cover.

Users act on documentation. They choose parameters by it, interpret their results by it, and copy its description of the method into the methods sections of their publications. Missing documentation is easy to notice; wrong documentation is not. A description of last year's default, or of the method as a paper published it before the code changed, gives no error, and its consequences show up far away, in a result reported with the wrong method. Two ideas run through this guide. Readers come to documentation with different needs, and each kind of document serves one (Section 1). And documentation stays true only if it lives, changes, and is checked together with the code (Section 3). Section 2 is about writing the reference, the comments, the changelog, and the citation.

What has to be documented belongs to *Design Principles*, because it is part of what the software promises. Its principles require, among other things, the method with its assumptions, defaults, and references (*Design Principles*, "Document the method, not the implementation"), the policy for

missing values (*Design Principles*, “Mind the gap, don’t fill it”), the time and memory complexity (*Design Principles*, “Size matters”), a changelog (*Design Principles*, “Design stable APIs”), and citation metadata (*Design Principles*, “Make every result traceable to a version”). This guide says how such documentation is organized, written, and kept true. Documentation explains correct use, but it does not enforce it: users who never read it are protected only by the checks of the code (*Design Principles*, “Garbage in, useful error out”).

Normative language and the ladder

The words *must*, *should*, and *may*, and the plain imperatives “prefer” and “avoid”, carry the meaning defined in the subsection “Normative language” of *Design Principles*. The rules of this guide apply along the same “Ladder of Research Software”: the *musts* are part of the floor from the second rung on, and what rises with the rung is how much documentation there is and for whom it is written (Subsection 1.2). When rules conflict, the priorities in *Design Principles*, “Get your priorities right” decide. Each rule is stated in one of the documents only.

Terminology

The terms defined in *Design Principles*, such as package, library code, workflow code, entry point, user-facing function, host stack, and caller, carry the same meaning here, and so do the terms of testing defined in *Types of Research Software Tests*, such as test, oracle, and snapshot. This guide adds the four kinds of documentation that Subsection 1.1 keeps apart:

Tutorial A lesson that takes a newcomer through a complete task, step by step, so that they learn to use the software.

How-to guide Directions for one task that a user who already knows the software wants to get done.

Reference The description of every entry point, option, and file format, made for looking things up.

Explanation The discussion of a topic, such as the method, its assumptions, and the reasons for a design, made for understanding.

At a glance

Every principle of this guide in one line. The number and the title link to the principle.

1 What to document		
1.1	Diátaxis: tutorials, how-to guides, reference, explanation	Give each document one purpose, and provide the kinds that the readers need.
1.2	Grow the documentation with the software	Write what the rung demands, from a README on the second rung to documentation that users can work from alone.
1.3	The README is the front door	Say what the software does, its status, how to install, run, and cite it, and where to go next.
1.4	The paper is not the manual	Cite the paper, but document the method as the current version implements it, and every difference.
2 Writing documentation		
2.1	The reference lives next to the code	Write the reference in the source, next to the code it describes, and generate it from there.
2.2	Document every key and every file	Document every configuration key, every input and output file, and the resources a run needs.
2.3	Comments say why, not what	Explain reasons, sources, and conventions in comments, and record design decisions in the repository.
2.4	Keep a changelog	Write the changelog for users, by release and kind of change, with changes of results set apart.
2.5	Credit where credit is due	Say how to cite the software, the method, and the tools it runs, in every place users look.
3 Keeping documentation true		
3.1	Docs as code	Keep the documentation in the repository, change it with the code, build it on every change, and publish it per version.
3.2	Every example runs	Run every example on every change, and check the output it shows.

1 What to document

1.1 Diátaxis: tutorials, how-to guides, reference, explanation

Readers come to documentation with one of four needs: to learn the software, to get a task done, to look something up, or to understand. The Diátaxis framework assigns each need a kind of document: a tutorial, a how-to guide, a reference, and an explanation ([Terminology](#)). Each document should serve one of these needs. A document that serves several serves none well: a tutorial that stops to list every option loses the newcomer, and a reference interrupted by derivations makes a reader who came to look something up search for it.

In research software, the four kinds take typical forms:

- **Tutorial.** A complete analysis on a small dataset that ships with the package or is generated, from the input to the result, with the output shown at each step.
- **How-to guide.** Directions for a task such as running on a cluster, using one's own reference genome or grid, reading another file format, or reproducing the analysis of a paper.
- **Reference.** The description of every user-facing function, command, configuration key, and file format (Subsections [2.1](#) and [2.2](#)).
- **Explanation.** The method: its derivation and assumptions, when it applies and when it does not, how it compares with the alternatives, and how it was validated (Subsection [1.4](#)).

Keep them apart, and link them. A tutorial links to the reference for the functions it uses, and the reference of a function links to the explanation of its method instead of repeating it (*Design Principles*, “DRY: Don’t repeat yourself”). Which of the four a package needs, and how much of each, depends on its rung (Subsection 1.2).

1.2 Grow the documentation with the software

Documentation costs effort to write and more to keep true, and it should grow with the rung of the software on the “Ladder of Research Software” (*Design Principles*, “Pay as you grow”). Write the documentation that the readers of the current rung need, and add the rest when the software climbs, before new readers arrive.

On the first rung, nothing is required, but a few lines on how to run the code and what it assumes help its author a year later. From the second rung on, the floor of *Design Principles* already requires much of the reference: the method, the policy for missing values, and the other parts listed in [About this guide](#); a README (Subsection 1.3) makes it usable by colleagues who can still ask the authors. From the third rung on, users cannot ask, and the ladder asks for documentation that they can work from alone: a complete reference, at least one tutorial, how-to guides for the common tasks, and documentation published for each version (Subsection 3.1). On the fourth rung, the explanation of the method, its limits, and its validation should be as thorough as the cost of a wrong result demands.

Documentation that has gone out of date is worse than none, because readers trust it. Do not write more than can be kept true: a short README that is right serves users better than a long manual that describes an earlier version.

1.3 The README is the front door

Every package from the second rung on should have a README, and it should answer, in this order, the questions of someone who has just found the package:

- What it does, in one or two sentences, and for which field and data.
- Its status: the rung, for example whether it is maintained for others or shared as is, and the stability commitment (*Design Principles*, “Design stable APIs”). Where the package deviates from a *should* because of its rung, the subsection “Normative language” of *Design Principles* asks for the reason to be stated once for the whole package, and the README is the place.
- How to install it, including the supported versions of the language and of the host stack.
- A minimal example: the shortest complete use, on data that the reader has or the package provides.
- How to cite it (Subsection 2.5), its license, and how to get help and report bugs.
- Where the full documentation is.

Keep the README short. It is the front door, not the house: it points to the tutorials and the reference rather than repeating them, because every repetition is a copy that can drift (*Design Principles*, “DRY: Don’t repeat yourself”). The minimal example is an example like any other and runs on every change (Subsection 3.2).

1.4 The paper is not the manual

Much research software comes with a paper that describes its method, and the paper should be cited from the documentation (*Design Principles*, “Document the method, not the implementation”). It cannot replace the documentation. A paper describes the method as it was at the time of publication, for reviewers rather than users, and it often leaves out what users need: the defaults, the treatment of edge cases, and the options added later. The paper is fixed, and the software moves on.

The documentation should therefore describe the method as the current version implements it. *Design Principles* requires it to state every difference from the literature it cites (*Design Principles*, “Document the method, not the implementation”), such as a changed default, a corrected formula, an approximation, or an extension. Give each difference with the version in which it appeared: users who cite the paper for their results need it to report what was actually computed (*Design Principles*, “Make every result traceable to a version”). Where the method of a paper is meant to be reproduced exactly, say which version of the software reproduces it, and keep that version available.

2 Writing documentation

2.1 The reference lives next to the code

Write the reference of each user-facing function in the source, next to its code, in the host stack’s format for documentation comments, such as docstrings in Python, roxygen comments in R, or docstrings in Julia, and generate the published reference from there with the host stack’s tools, such as Sphinx, pkgdown, or Documenter. A reference written in a separate place drifts from the code it describes, and nobody notices; one written next to the code is in view whenever the code changes, and in the same change (Subsection 3.1).

The reference of a user-facing function should state what it computes, each parameter with its type, shape, unit, and default, the return value with the same, the errors and warnings it raises, and a short example (Subsection 3.2). For library code, the meaning of each parameter matters more than its type, which the signature often shows already: not “tolerance, a float”, but which error it bounds, whether it is relative or absolute, and what happens when it is not met.

Where the tools allow it, generate what the reference states from the code, such as the default values from the signature and the list of choices from the code that validates them, rather than writing it a second time (*Design Principles*, “DRY: Don’t repeat yourself”).

2.2 Document every key and every file

The reference of workflow code is not its functions but what its users write and read: the command that runs it, the configuration, and the files that go in and come out. These form its public API (*Design Principles*, “Design stable APIs”), and each part should be documented:

- **Running it.** The command and its options, and the environment it needs (*Design Principles*, “Pin what you run”).
- **Configuration.** Every key, with its meaning, type, unit, default, and allowed values, and the keys that only make sense together.
- **Inputs.** Every input file and its format, including the reference data and the versions that were tested, and the table of samples or inputs with its columns.
- **Outputs.** Every output file, what it contains, its format and columns with their units, and which outputs are final and which are intermediate.
- **Resources.** The memory, the number of cores, the disk space, and the run time that each step needs, as functions of the size of the input (*Design Principles*, “Size matters”), so that users can request them on a cluster.

The configuration is documented best where it is defined: in a schema that both validates the configuration (*Design Principles*, “Garbage in, useful error out”) and generates its reference, so that the two cannot disagree (*Design Principles*, “DRY: Don’t repeat yourself”). The small dataset of the end-to-end test (*Design Principles*, “Test the whole workflow on a small dataset”) doubles as the dataset of a tutorial: a run that users can start right after installing, with outputs they can compare with the documented ones.

2.3 Comments say why, not what

The code says what it does; a comment should say what the code cannot: why it does it that way. Good comments give the reason for a choice that a reader would otherwise undo, such as a reordering that avoids cancellation or a loop kept for memory; the source of a formula, down to the equation number and the sign convention of the paper; the unit or convention of a value where its name does not say it (*Design Principles*, “Name axes, units, and conventions”); and the bug report behind a workaround, so that it can be removed once the bug is fixed. Comments that repeat the code add nothing and fall out of date with the first change, and from then on they contradict it.

Comments explain a few lines; some knowledge concerns the whole package. Decisions about its design, such as why a method was chosen over the alternatives, why a dependency was added, or why a convention differs from the host stack’s, should be recorded in the repository when they are made, for example as short decision records, together with the conventions and the release process. They are the documentation of the maintainers, and without them the package depends on the memory of its authors (*Design Principles*, “Code is for life, not just for Christmas”).

2.4 Keep a changelog

Design Principles requires a changelog that records every change of behavior (*Design Principles*, “Design stable APIs”). Write it for users, not for the developers: a list of commit messages says what was done to the code, while users need to know what changed for them and whether they have to act. The format of Keep a Changelog serves this well: one entry per release, with its version and date, and the changes grouped by kind, such as added, changed, deprecated, removed, and fixed.

Changes that can alter results should stand in a group of their own, such as “changes of results”, rather than among the fixes, so that users can decide at a glance whether to rerun an analysis. Each such entry says which entry points are affected, how the results change, and how to get the old behavior back if that is possible. Each deprecation names its replacement and the release in which the old behavior will be removed.

2.5 Credit where credit is due

Design Principles requires published software to provide machine-readable citation metadata (*Design Principles*, “Make every result traceable to a version”). Make citing it easy as well: users cite what they find quickly. Software that is not cited is not credited, and a result that does not cite it cannot be traced to it.

State how to cite the software in every place users look: the README, the documentation, and the host stack’s citation mechanism, such as the `citation` function of R; a `CITATION.cff` file in the repository is read by code-hosting platforms and archives. Ask for two citations where there are two contributions, the software and the paper of the method (Subsection 1.4). Archives such as Zenodo give each release a DOI of its own and the software a DOI for all versions: ask users to cite the DOI of the version they ran, so that the citation also says which version computed the result.

Software that runs the methods of others, such as a workflow that calls established tools, should also say which of them to cite. Workflow code can list them next to the versions of the tools that it records for each run (*Design Principles*, “Make every result traceable to a version”).

3 Keeping documentation true

3.1 Docs as code

Documentation should be treated like code: kept in the repository of the package, written in plain-text formats such as Markdown, reStructuredText, or Quarto, reviewed, built, and released with the code.

Documentation kept elsewhere, such as in a wiki, a shared drive, or a slide deck, has no version, changes on its own schedule, and describes some version of the software, but not a stated one.

A change of behavior and the change of its documentation belong in the same commit, and the review of a change checks both. When the code and the documentation disagree, one of them is wrong, and the disagreement is a bug like any other: decide which describes the intended behavior, and fix the other.

Build the documentation automatically on every change, and treat warnings, such as broken links, references that do not resolve, and user-facing functions without documentation, as errors. Publish it for each release, so that readers can choose the documentation of the version they run, and show the version on every page (*Design Principles*, “Make every result traceable to a version”). The documentation of the development version may be published too, but it should not be the only one, because it describes features that the released version does not have.

3.2 Every example runs

Every example in the documentation should run automatically on every change: the examples of the reference, the minimal example of the README, the tutorials, and the how-to guides. An example that is never run breaks silently when the code changes, and a reader who copies it gets an error at best and a wrong result at worst. Host stacks provide the means, such as doctests in Python and Julia, the examples and vignettes that the check of an R package runs, and notebooks and Quarto documents executed when the documentation is built.

Running an example shows that it runs; comparing its printed output with the documented output, as a doctest does, shows that the output has not changed (*Types of Research Software Tests*, “It runs”; *Types of Research Software Tests*, “Snapshot”). Neither shows that the result is right, so examples do not replace tests (*Design Principles*, “Testing is not optional”). Yet readers take the output that an example shows as the right answer, so check it against an oracle before publishing it, as for any reference output (*Testing Research Software*, “Unchanged is not correct”). Printed floating-point numbers vary in their last digits between platforms: print fewer digits, or use the tolerance options of the doctest tool, rather than dropping the output.

Examples need data that are always there. Use small datasets that ship with the package or are generated from a fixed seed. Where an example needs real data that are too large to ship, download them from an archive with a persistent identifier, check them against a stored checksum, and keep a copy for the build, so that the documentation does not break when a server moves.